

Java For SDET

Part 1: Core & Must Know

★ 15 Core Topics for SDET Interviews ★



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

Java Basics	OOPS Concepts	Constructors	Strings
Arrays	Collections Framework	Exception Handling	Multithreading
Java 8 Features	Streams API	Functional Interfaces	Lambda Expressions
Comparable vs Comparator	HashMap Internal Working	JVM vs JRE vs JDK	

Introduction

This document is **Part 1** of the Java For SDET Complete Guide. It covers the **15 Core & Must-Know topics** that are asked in almost every SDET interview — from product companies to service-based interviews.

These topics form the foundation of your Java knowledge as an SDET. Master these before moving to Part 2.

What you will learn	Why it matters
Java Basics & OOPS	Foundation for all interviews
Strings, Arrays, Collections	Asked in every Java round
Exception Handling & Multithreading	Framework design essential
Java 8: Streams, Lambda, Functional	Modern Java — must know
HashMap & Comparable/Comparator	Collections deep dive

Table of Contents

PART 1: CORE & MUST KNOW JAVA FOR SDET

1. Java Basics ★
2. OOPS Concepts ★
3. Constructors ★
4. Strings ★
5. Arrays ★
6. Collections Framework ★
7. Exception Handling ★
8. Multithreading ★
9. Java 8 Features ★
10. Streams API ★
11. Functional Interfaces ★
12. Lambda Expressions ★
13. Comparable vs Comparator ★
14. HashMap Internal Working ★
15. JVM vs JRE vs JDK ★

JAVA BASICS

1. Java Basics ★

Overview

Java is the foundation of all Java interview preparation. Interviewers check core Java understanding, programming fundamentals, and framework readiness.

Topics Covered

- What is Java
- Features of Java ★
- JVM vs JDK vs JRE ★
- Java Program Structure
- Variables & Data Types
- Operators
- Conditional Statements
- Loops
- Methods in Java ★
- Class & Object Basics
- Static Keyword Basics
- Common Interview Questions

► What is Java

What is it: Java is an object-oriented, platform-independent programming language used for backend, automation, enterprise, and web applications.

Key Components:

- Object-oriented
- Platform independent
- Secure
- Robust
- Portable

How YOU should answer: Java is a high-level, object-oriented programming language widely used for enterprise applications, automation frameworks, backend systems, and web development.

```
Java

public class Main {

    public static void main(String[] args) {
```

```
System.out.println("Hello Kalpesh");  
}  
}
```

► Features of Java

What is it: Core characteristics that make Java powerful and popular.

Key Components:

- Platform Independent
- Object-Oriented
- Secure
- Robust
- Multithreaded
- Portable

How YOU should answer: Java is popular because it is object-oriented, platform-independent, secure, robust, and supports multithreading.

Java

Write Once → Run Anywhere

► JVM vs JDK vs JRE

What is it: Core Java runtime architecture components.

Key Components:

- JVM — Runs Java bytecode
- JRE — JVM + libraries to run Java
- JDK — JRE + development tools

How YOU should answer: JVM executes Java bytecode, JRE provides runtime environment, and JDK provides development tools to build Java applications.

Flow

.java

↓

Compiler

↓

.class (Bytecode)

↓

JVM Executes

► Variables & Data Types

What is it: Variables store data values in memory.

Key Components:

- Primitive: int, double, char, boolean
- Non-Primitive: String, Arrays, Objects

How YOU should answer: Variables are memory containers used to store values, and Java supports primitive and non-primitive data types.

```
Java
int age = 25;
String name = "Kalpesh";
```

► Methods in Java

What is it: A method is a block of code that performs a specific task and can be reused.

Key Components:

- Return type
- Method name
- Parameters
- Method body

How YOU should answer: Methods are reusable blocks of code that improve readability, reusability, and separation of logic.

```
Java
public int add(int a, int b) {
    return a + b;
}
```

Common Mistakes

- ✗ Saying only 'Java is programming language'
- ✗ Forgetting platform independence
- ✗ Not mentioning automation use case

Expected Interview Questions

- What is Java?
- Why Java is platform independent?
- Difference between JVM, JDK, and JRE?
- Why Java is widely used in automation?

OOPS CONCEPTS

2. OOPS Concepts ★

Overview

OOPS is one of the MOST IMPORTANT Java interview topics. Asked in every Java round, framework round, automation architecture round, and coding round.

Topics Covered

- What is OOPS
- Four Pillars of OOPS ★
- Encapsulation ★
- Inheritance ★
- Polymorphism ★
- Abstraction ★
- Class vs Object
- Method Overloading vs Overriding ★
- Constructor Basics
- Real-Time Framework Examples ★
- Common Mistakes
- Common Interview Questions

► What is OOPS

What is it: OOPS (Object-Oriented Programming System) is a programming approach based on objects and classes.

Key Components:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

How YOU should answer: OOPS is a programming paradigm that organizes code using classes and objects to improve reusability, maintainability, and scalability.

```
Java
```

```
class Car {  
    String color;  
}
```

► Four Pillars of OOPS

What is it: Core principles of object-oriented programming.

Key Components:

- 1. Encapsulation
- 2. Inheritance
- 3. Polymorphism
- 4. Abstraction

How YOU should answer: The four pillars of OOPS are Encapsulation, Inheritance, Polymorphism, and Abstraction which help build maintainable code.

➤ **Encapsulation**

What is it: Wrapping data and methods inside a class and restricting direct access.

Key Components:

- Private variables
- Getter/Setter methods
- Data hiding

How YOU should answer: Encapsulation hides internal data using private variables and provides controlled access through getters and setters.

Java

```
class Person {  
    private String name;  
    public String getName() { return name; }  
}
```

➤ **Inheritance**

What is it: A child class inherits properties and methods from parent class.

Key Components:

- extends keyword
- Parent-child relationship
- Code reusability

How YOU should answer: Inheritance allows a child class to reuse properties and methods of parent class, improving code reusability.

Java

```
class Animal {  
    void eat() { System.out.println("Eating"); }  
}  
  
class Dog extends Animal {}
```


► Polymorphism

What is it: Same method behaves differently based on context.

Key Components:

- Compile-time (Overloading)
- Runtime (Overriding)

How YOU should answer: Polymorphism allows one interface to have multiple implementations — overloading at compile time and overriding at runtime.

```
Java
// Overloading
void add(int a) {}
void add(int a, int b) {}
```

► Abstraction

What is it: Hiding implementation details and showing only functionality.

Key Components:

- Abstract class
- Interface
- Data hiding

How YOU should answer: Abstraction hides internal implementation and exposes only necessary functionality using abstract classes and interfaces.

```
Java
abstract class Vehicle {
    abstract void start();
}
```

► Method Overloading vs Overriding

What is it: Two types of polymorphism in Java.

Key Components:

- Overloading — same name, different parameters (compile-time)
- Overriding — same name, same parameters in child class (runtime)

How YOU should answer: Overloading is compile-time polymorphism with same method name but different parameters. Overriding is runtime polymorphism in child class.

```
Java
// Overriding
@Override
```

```
void start() { System.out.println("Car starts"); }
```

Common Mistakes

- ✗ Saying only theory without real-time examples
- ✗ Confusing overloading and overriding
- ✗ Not relating OOPS to automation framework

Expected Interview Questions

- What is OOPS?
- Four pillars of OOPS?
- Difference between overloading and overriding?
- How OOPS is used in automation frameworks?
- Difference between abstract class and interface?

★ OOPS in Automation Frameworks

1. Encapsulation → Page Object Model
2. Inheritance → Base class reuse
3. Polymorphism → Driver management
4. Abstraction → Framework layers

CONSTRUCTORS

3. Constructors ★

Overview

One of the MOST ASKED OOPS interview topics. Especially important for object creation, OOPS understanding, and constructor chaining concepts.

Topics Covered

- What is Constructor
- Types of Constructors ★
- Default Constructor ★
- Parameterized Constructor ★
- Constructor Overloading ★
- Constructor Chaining ★

- this() vs super() ★
- Private Constructor ★
- Real-Time Framework Usage ★
- Common Mistakes
- Common Interview Questions

► What is Constructor

What is it: Constructor is a special method used to initialize objects.

Key Components:

- Same name as class
- No return type
- Executes automatically on object creation

How YOU should answer: Constructor is a special method automatically invoked during object creation for initialization.

Java

```
class Student {  
    Student() {  
        System.out.println("Constructor");  
    }  
}
```

► Types of Constructors

What is it: Java supports Default and Parameterized constructors.

Key Components:

- Default Constructor — no parameters
- Parameterized Constructor — accepts arguments
- Copy Constructor — copies object values

How YOU should answer: Java mainly supports default and parameterized constructors.

Java

```
// Default  
Student() {}  
  
// Parameterized  
Student(String name) { this.name = name; }
```

► Constructor Chaining

What is it: Calling one constructor from another using this() or super().

Key Components:

- `this()` — calls same class constructor
- `super()` — calls parent class constructor

How YOU should answer: Constructor chaining allows one constructor to call another to reduce duplicate initialization code.

Java

```
Student() {  
    this("Default");  
}  
  
Student(String name) {  
    this.name = name;  
}
```

➤ Private Constructor

What is it: Restricts object creation from outside the class.

Key Components:

- Singleton pattern
- Utility classes
- Controlled object creation

How YOU should answer: Private constructors are used in Singleton design pattern to ensure only one instance is created.

Java

```
class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if(instance == null) instance = new Singleton();  
        return instance;  
    }  
}
```

➤ Real-Time Framework Usage

What is it: Constructors are used heavily in automation frameworks.

Key Components:

- Page Object initialization
- Driver setup

- Base class initialization

How YOU should answer: In POM framework, constructors initialize page locators and driver references for each page class.

```
Java

class LoginPage {
    WebDriver driver;
    LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }
}
```

Common Mistakes

- ✗ Adding return type to constructor
- ✗ Confusing method and constructor
- ✗ Not knowing constructor chaining

Expected Interview Questions

- What is constructor?
- Constructor vs method?
- What is constructor chaining?
- Why use private constructor?
- this() vs super()?

STRINGS

4. Strings ★

Overview

One of the MOST ASKED Java interview topics. String vs StringBuilder, String Pool, and Immutable concept are very important.

Topics Covered

- What is String
- String Immutability ★
- String Pool ★

- String Creation Methods
- String vs StringBuilder vs StringBuffer ★
- Common String Methods
- == vs equals() ★
- StringBuilder ★
- Real-Time Framework Usage ★
- Common Mistakes
- Common Interview Questions

► What is String

What is it: String is an immutable class in Java used to store and manipulate textual data.

Key Components:

- Immutable
- Sequence of characters
- Stored in String Pool

How YOU should answer: String is an immutable class in Java used to store textual data efficiently using String Pool.

```
Java
String name = "Kalpesh";
String name2 = new String("Kalpesh");
```

► String Immutability

What is it: Once a String object is created, its value cannot be changed.

Key Components:

- Security
- Caching
- Thread safety
- Memory optimization

How YOU should answer: Strings are immutable in Java to improve security, caching, synchronization, and performance.

```
Java
String s = "Hello";
s = s + " World"; // New object created
```

► String Pool

What is it: A special memory area in Java Heap where String literals are stored.

Key Components:

- Memory optimization
- Literal reuse
- Heap vs String Pool

How YOU should answer: String Pool stores String literals to avoid duplicate objects and improve memory efficiency.

```
Java
String a = "Java"; // String Pool
String b = "Java"; // Same reference
System.out.println(a == b); // true
```

► String vs StringBuilder vs StringBuffer

What is it: Three ways to work with text in Java with different characteristics.

Key Components:

- String — Immutable, thread-safe
- StringBuilder — Mutable, NOT thread-safe, fast
- StringBuffer — Mutable, thread-safe, slower

How YOU should answer: StringBuilder is preferred for string manipulation in single-threaded scenarios for performance.

```
Java
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Kalpesh");
System.out.println(sb.toString());
```

► == vs equals()

What is it: == compares references, equals() compares content.

Key Components:

- == — reference comparison
- equals() — value/content comparison

How YOU should answer: == checks if two references point to same object, while equals() checks actual string content.

```
Java
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

Common Mistakes

- ✗ Confusing mutable and immutable
- ✗ Using == for string comparison
- ✗ Ignoring String Pool

Expected Interview Questions

- Why String is immutable?
- == vs equals()?
- String vs StringBuilder?
- What is String Pool?
- Why StringBuilder is faster?

★ Most Important String Topics

1. String Immutability
2. String Pool
3. == vs equals()
4. String vs StringBuilder vs StringBuffer

ARRAYS

5. Arrays ★

Overview

One of the MOST IMPORTANT Java coding topics for SDET interviews. Important for problem solving, Collections understanding, and data handling.

Topics Covered

- What is Array
- Why Arrays are Used ★
- Types of Arrays
- Array Declaration & Initialization
- Traversing Arrays ★
- Single vs Multi-Dimensional Arrays ★
- Common Array Operations
- Arrays vs ArrayList ★

- Real-Time Framework Usage ★

► What is Array

What is it: Array is a fixed-size data structure used to store multiple values of the same datatype in contiguous memory.

Key Components:

- Fixed size
- Indexed access
- Same datatype
- Contiguous memory

How YOU should answer: Array is a fixed-size data structure used to store multiple values of the same datatype efficiently using indexes.

```
Java
int[] numbers = {1, 2, 3, 4};
String[] names = {"Kalpesh", "Java"};
```

► Traversing Arrays

What is it: Accessing each element of an array using loops.

Key Components:

- for loop
- for-each loop
- index-based access

How YOU should answer: Arrays are traversed using for loops or enhanced for-each loops for accessing individual elements.

```
Java
int[] arr = {1, 2, 3};
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

► Single vs Multi-Dimensional Arrays

What is it: Single arrays store one row, multi-dimensional store rows and columns.

Key Components:

- 1D — `int[] arr = new int[5]`
- 2D — `int[][] matrix = new int[3][3]`

How YOU should answer: Single-dimensional arrays store one set of values while multi-dimensional arrays store matrix-like data.

Java

```
int[][] matrix = {{1,2},{3,4}};  
System.out.println(matrix[0][1]); // 2
```

► Arrays vs ArrayList

What is it: Arrays are fixed size, ArrayList is dynamic.

Key Components:

- Array — fixed size, faster
- ArrayList — dynamic, slower, more features

How YOU should answer: Arrays have fixed size and better performance while ArrayList provides dynamic resizing and utility methods.

Java

```
// Array  
int[] arr = new int[5];  
  
// ArrayList  
ArrayList<Integer> list = new ArrayList<>();  
list.add(1);
```

Common Mistakes

- ✗ Index out of bounds errors
- ✗ Using arrays when ArrayList is better
- ✗ Not knowing array limitations

Expected Interview Questions

- What is array?
- Advantages of arrays?
- Difference between array and ArrayList?
- Real-time array usage?

COLLECTIONS FRAMEWORK

6. Collections Framework ★

Overview

One of the MOST IMPORTANT Java topics for SDET interviews. Asked in Java rounds, coding rounds, framework design rounds, and automation architecture discussions.

Topics Covered

- What is Collections Framework
- Why Collections are Used ★
- List Interface ★
- Set Interface ★
- Queue Interface
- Map Interface ★
- ArrayList ★
- LinkedList
- HashSet
- HashMap ★
- Difference Between Collection Types ★
- Common Interview Questions

► What is Collections Framework

What is it: Java Collections Framework is a set of classes and interfaces for storing and manipulating groups of objects.

Key Components:

- List
- Set
- Queue
- Map

How YOU should answer: Collections Framework provides ready-made data structures like List, Set, Queue, and Map for efficient data storage and manipulation.

```
Java
List<String> list = new ArrayList<>();
Set<String> set = new HashSet<>();
Map<String,Integer> map = new HashMap<>();
```

► List Interface

What is it: Ordered collection that allows duplicates.

Key Components:

- ArrayList — dynamic array
- LinkedList — doubly linked list
- Ordered

- Allows duplicates

How YOU should answer: List maintains insertion order, allows duplicates, and provides index-based access.

Java

```
List<String> list = new ArrayList<>();
list.add("Kalpesh");
list.add("Java");
System.out.println(list.get(0));
```

► Set Interface

What is it: Unordered collection that does NOT allow duplicates.

Key Components:

- HashSet — no order
- LinkedHashSet — insertion order
- TreeSet — sorted order

How YOU should answer: Set does not allow duplicate values and is used for unique data storage.

Java

```
Set<String> set = new HashSet<>();
set.add("Java");
set.add("Java"); // Ignored
```

► Map Interface

What is it: Stores key-value pairs where keys are unique.

Key Components:

- HashMap — no order
- LinkedHashMap — insertion order
- TreeMap — sorted keys

How YOU should answer: Map stores data as key-value pairs where each key is unique.

Java

```
Map<String,Integer> map = new HashMap<>();
map.put("Kalpesh", 1);
System.out.println(map.get("Kalpesh"));
```

Common Mistakes

- ✗ Confusing List and Set
- ✗ Not knowing HashMap behavior

✗ Using wrong collection type

Expected Interview Questions

- What is Collections Framework?
- Difference between List, Set, and Map?
- When to use ArrayList vs LinkedList?
- Why HashMap is widely used?

Collection	Order	Duplicates	Null
ArrayList	Insertion order	Yes	Yes
LinkedList	Insertion order	Yes	Yes
HashSet	No order	No	One null
TreeSet	Sorted	No	No
HashMap	No order	Keys unique	One null key

EXCEPTION HANDLING

7. Exception Handling ★

Overview

One of the MOST IMPORTANT Java topics for SDET interviews. Asked in Java rounds, framework design discussions, automation debugging rounds, and real-time scenario discussions.

Topics Covered

- What is Exception Handling
- Why Exception Handling is Important ★
- Types of Exceptions ★
- Checked vs Unchecked Exceptions ★
- try-catch Block
- finally Block ★
- throw vs throws ★
- Custom Exceptions
- Exception Handling in Frameworks ★
- Real-Time Scenarios
- Common Mistakes
- Common Interview Questions

► What is Exception Handling

What is it: Exception handling is a mechanism to handle runtime errors gracefully without crashing the program.

Key Components:

- try
- catch
- finally
- throw
- throws

How YOU should answer: Exception handling allows programs to catch and handle runtime errors gracefully using try-catch-finally blocks.

```
Java

try {
    int result = 10/0;
} catch(ArithmeticException e) {
    System.out.println("Error: "+e.getMessage());
} finally {
    System.out.println("Always executes");
}
```

► Checked vs Unchecked Exceptions

What is it: Two categories of exceptions in Java.

Key Components:

- Checked — compile-time (IOException, SQLException)
- Unchecked — runtime (NullPointerException, ArrayIndexOutOfBoundsException)

How YOU should answer: Checked exceptions are caught at compile time while unchecked exceptions occur at runtime.

```
Java

// Checked
try { FileReader f = new FileReader("file.txt"); }
catch(IOException e) {}

// Unchecked
String s = null;
s.length(); // NullPointerException
```

► throw vs throws

What is it: throw is used to throw an exception, throws declares possible exceptions.

Key Components:

- throw — inside method to throw exception
- throws — in method signature to declare exceptions

How YOU should answer: throw explicitly throws an exception while throws in method signature declares that method might throw exceptions.

```
Java
// throw
throw new IllegalArgumentException("Invalid");

// throws
public void readFile() throws IOException {}
```

► Custom Exceptions

What is it: User-defined exception classes for specific business scenarios.

Key Components:

- extends Exception or RuntimeException
- Custom message
- Business-specific errors

How YOU should answer: Custom exceptions help handle specific business errors with meaningful messages.

```
Java
class InvalidTestDataException extends RuntimeException {
    InvalidTestDataException(String msg) { super(msg); }
}
```

► Exception Handling in Frameworks

What is it: Exception handling is critical for stable automation frameworks.

Key Components:

- StaleElementReferenceException
- NoSuchElementException
- TimeoutException

How YOU should answer: In automation frameworks, exception handling prevents test suite failures due to unexpected element or timing issues.

```
Java
```

```
try {  
    driver.findElement(By.id("btn")).click();  
} catch(NoSuchElementException e) {  
    logger.error("Element not found");  
}
```

Common Mistakes

- ✗ Empty catch blocks
- ✗ Catching generic Exception always
- ✗ Not using finally for cleanup

Expected Interview Questions

- What is exception handling?
- Checked vs unchecked exceptions?
- throw vs throws?
- How do you handle exceptions in automation frameworks?
- What is custom exception?

MULTITHREADING

8. Multithreading ★

Overview

One of the MOST IMPORTANT advanced Java topics for SDET interviews. Especially for Selenium Grid, Parallel Testing, and Framework optimization.

Topics Covered

- What is Multithreading
- Process vs Thread ★
- Thread Lifecycle
- Creating Threads ★
- Runnable Interface ★
- Thread Methods
- Synchronization ★
- Race Condition
- Deadlock Basics
- Multithreading in Automation ★

► What is Multithreading

What is it: Multithreading allows concurrent execution of two or more threads to maximize CPU utilization.

Key Components:

- Thread
- Concurrency
- Parallel execution
- CPU optimization

How YOU should answer: Multithreading enables running multiple threads simultaneously for better performance and resource utilization.

```
Java
// Thread 1 and Thread 2 running simultaneously
```

► Process vs Thread

What is it: Process is an independent program, Thread is a lightweight unit within a process.

Key Components:

- Process — independent memory, heavy
- Thread — shared memory, lightweight

How YOU should answer: A process is a heavyweight independent program while a thread is a lightweight unit that shares process memory.

► Creating Threads

What is it: Two ways to create threads in Java.

Key Components:

- Extending Thread class
- Implementing Runnable interface

How YOU should answer: Threads are created either by extending Thread class or implementing Runnable interface.

```
Java
// Thread class
class MyThread extends Thread {
    public void run() { System.out.println("Thread"); }
}

// Runnable
class MyTask implements Runnable {
    public void run() { System.out.println("Task"); }
```

```
}
```

► Synchronization

What is it: Controls access to shared resources to prevent race conditions.

Key Components:

- synchronized keyword
- Thread-safe access
- Monitor lock

How YOU should answer: Synchronization ensures only one thread accesses shared resources at a time to prevent data inconsistency.

```
Java  
  
synchronized void increment() {  
    count++;  
}
```

► Multithreading in Automation

What is it: Parallel test execution uses multithreading for faster automation.

Key Components:

- Parallel execution
- Selenium Grid
- TestNG parallel mode

How YOU should answer: Automation frameworks use multithreading for parallel test execution across browsers and environments.

```
Java  
  
// TestNG parallel execution  
@Test  
// Runs in parallel threads
```

Common Mistakes

- ✗ Creating too many threads
- ✗ Not handling race conditions
- ✗ Ignoring thread safety in frameworks

Expected Interview Questions

- What is multithreading?
- Process vs Thread?
- How do you create threads?

- What is synchronization?
- How multithreading is used in automation?

JAVA 8 FEATURES

9. Java 8 Features ★

Overview

One of the MOST IMPORTANT Java interview topics for modern SDET roles. Especially Streams, Lambda, and Functional Interfaces. Asked in product companies and framework discussions.

Topics Covered

- What is Java 8
- Features of Java 8 ★
- Lambda Expressions ★
- Functional Interfaces ★
- Streams API ★
- forEach Method
- Method References ★
- Optional Class ★
- Default Methods ★

► What is Java 8

What is it: Java 8 introduced major features that modernized Java with functional programming capabilities.

Key Components:

- Lambda Expressions
- Streams API
- Functional Interfaces
- Optional Class
- Default Methods
- Method References

How YOU should answer: Java 8 brought functional programming to Java with Lambda expressions, Streams, and Functional Interfaces.

► Lambda Expressions

What is it: Short, anonymous functions that implement functional interfaces.

Key Components:

- Functional interface required
- Arrow operator ->
- Reduced boilerplate

How YOU should answer: Lambda expressions provide a concise way to implement functional interfaces without anonymous class syntax.

```
Java

// Without Lambda
Runnable r = new Runnable() {
    public void run() { System.out.println("Old"); }
};

// With Lambda
Runnable r = () -> System.out.println("Lambda");
```

► Streams API

What is it: Process collections of data in a functional, pipeline-based style.

Key Components:

- filter()
- map()
- collect()
- reduce()
- forEach()

How YOU should answer: Streams API allows processing collections using functional operations like filter, map, and collect.

```
Java

List<String> names = Arrays.asList("Kalpesh", "Java", "SDET");
names.stream()
    .filter(n -> n.startsWith("K"))
    .forEach(System.out::println);
```

► Default Methods

What is it: Methods with a body defined in interfaces, introduced in Java 8.

Key Components:

- interface method with body
- Backward compatibility
- default keyword

How YOU should answer: Default methods allow adding new methods to interfaces without breaking existing implementations.

```
Java

interface Vehicle {
    default void start() {
        System.out.println("Starting");
    }
}
```

► Method References

What is it: Shorthand for lambda expressions that call existing methods.

Key Components:

- ClassName::methodName
- instance::methodName

How YOU should answer: Method references simplify lambda expressions when calling existing methods directly.

```
Java

// Lambda
list.forEach(s -> System.out.println(s));

// Method Reference
list.forEach(System.out::println);
```

Common Mistakes

- ✗ Not knowing Java 8 features
- ✗ Confusing Lambda and anonymous class
- ✗ Weak Streams knowledge

Expected Interview Questions

- What are Java 8 features?
- What is Lambda expression?
- What is Streams API?
- What is functional interface?
- What are default methods?

★ Most Important Java 8 Topics

1. Lambda Expressions

2. Streams API
3. Functional Interfaces
4. Optional Class
5. Method References

STREAMS API

10. Streams API ★

Overview

One of the MOST IMPORTANT Java 8 interview topics. Important for collection processing, cleaner code, and functional programming.

Topics Covered

- What is Streams API
- Why Streams API is Important ★
- Stream Lifecycle ★
- Creating Streams
- Intermediate Operations ★
- Terminal Operations ★
- filter() ★
- map() ★
- collect() ★
- reduce()
- Real-Time Framework Usage ★
- Common Interview Questions

► What is Streams API

What is it: Streams API processes collections of objects in a functional, pipeline-based style.

Key Components:

- Source
- Intermediate operations
- Terminal operation

How YOU should answer: Streams API provides a declarative approach to process collections using filter, map, collect and other operations.

```
Java
list.stream()
  .filter(...)
  .map(...)
  .collect(Collectors.toList());
```

► Intermediate Operations

What is it: Operations that transform a stream and return another stream.

Key Components:

- filter() — filter elements
- map() — transform elements
- sorted() — sort elements
- distinct() — remove duplicates
- limit() — limit size

How YOU should answer: Intermediate operations are lazy and transform the stream without producing a final result.

```
Java
// filter
stream.filter(s -> s.startsWith("K"))

// map
stream.map(String::toUpperCase)
```

► Terminal Operations

What is it: Operations that produce a result and end the stream.

Key Components:

- collect() — gather into collection
- forEach() — iterate
- count() — count elements
- reduce() — combine elements

How YOU should answer: Terminal operations trigger stream processing and produce a final result.

```
Java
// collect
List<String> result = stream.collect(Collectors.toList());
```

```
// count
long count = stream.filter(s->s.length(>3).count();
```

► Real-Time Framework Usage

What is it: Streams are used in automation for data processing and filtering.

Key Components:

- Filter test data
- Process API responses
- Extract specific fields

How YOU should answer: Streams help process test data, filter API responses, and extract specific values in automation frameworks.

```
Java
// Filter active users from response
users.stream()
    .filter(u -> u.isActive())
    .collect(Collectors.toList());
```

Common Mistakes

- ✗ Not knowing difference between intermediate and terminal ops
- ✗ Reusing closed streams
- ✗ Ignoring lazy evaluation

Expected Interview Questions

- What is Streams API?
- Difference between intermediate and terminal operations?
- filter() vs map()?
- Real-time usage of Streams?

FUNCTIONAL INTERFACES

11. Functional Interfaces ★

Overview

One of the MOST IMPORTANT Java 8 topics for SDET interviews. Foundation for Lambda Expressions and Streams API.

Topics Covered

- What is Functional Interface
- Why Functional Interfaces are Important ★
- @FunctionalInterface Annotation ★
- Characteristics
- Predicate ★
- Function ★
- Consumer ★
- Supplier ★
- Real-Time Usage ★
- Common Interview Questions

► What is Functional Interface

What is it: An interface with exactly one abstract method used as the target for Lambda expressions.

Key Components:

- Single abstract method
- @FunctionalInterface annotation
- Lambda target

How YOU should answer: Functional Interface has exactly one abstract method and acts as the type for Lambda expressions.

```
Java
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

► Built-in Functional Interfaces

What is it: Java 8 provides ready-made functional interfaces.

Key Components:

- Predicate — returns boolean
- Function — transforms T to R
- Consumer — accepts T, returns nothing
- Supplier — supplies T value

How YOU should answer: Java 8 provides Predicate, Function, Consumer, and Supplier as standard functional interfaces.

```
Java
// Predicate
```

```
Predicate<String> isEmpty = s -> s.isEmpty();

// Function
Function<String,Integer> length = s -> s.length();

// Consumer
Consumer<String> print = s -> System.out.println(s);

// Supplier
Supplier<String> get = () -> "Kalpesh";
```

► Real-Time Usage

What is it: Used in Streams, event handling, and test data generation.

Key Components:

- Stream filter uses Predicate
- Event handling uses Consumer

How YOU should answer: Functional interfaces are used in Streams operations like filter (Predicate), map (Function), and forEach (Consumer).

```
Java

list.stream().filter(s -> s.length() > 3) // Predicate
.map(String::toUpperCase) // Function
.forEach(System.out::println); // Consumer
```

Common Mistakes

- ✗ Defining more than one abstract method
- ✗ Confusing functional interfaces with normal interfaces

Expected Interview Questions

- What is functional interface?
- Predicate vs Function?
- Why @FunctionalInterface annotation?
- How are functional interfaces used with Lambda?

LAMBDA EXPRESSIONS

12. Lambda Expressions ★

Overview

One of the MOST IMPORTANT Java 8 interview topics. Used for functional programming, cleaner code, and Streams API.

Topics Covered

- What is Lambda Expression
- Why Lambda Expressions are Important ★
- Lambda Syntax ★
- Functional Interface Connection ★
- Lambda with Parameters
- Lambda with Streams ★
- Real-Time Usage ★
- Common Interview Questions

► What is Lambda Expression

What is it: Lambda expression is a short anonymous function that implements a functional interface.

Key Components:

- Anonymous function
- Arrow operator ->
- Concise syntax
- Functional interface target

How YOU should answer: Lambda expressions provide a concise way to write anonymous functions without boilerplate anonymous class code.

Java

```
// Syntax: (parameters) -> expression

// No parameter
() -> System.out.println("Hello");

// One parameter
s -> System.out.println(s);

// Multiple parameters
(a, b) -> a + b;
```

► Lambda Syntax

What is it: Three forms of Lambda syntax based on parameters and body.

Key Components:

- No params: () -> body

- One param: `x -> body`
- Multiple params: `(x,y) -> body`
- Multi-line: `(x,y) -> { ... }`

How YOU should answer: Lambda syntax varies based on number of parameters and whether body is single expression or block.

```
Java
// Single expression
(a,b) -> a + b

// Block body
(a,b) -> {
    int sum = a+b;
    return sum;
}
```

► Lambda with Collections

What is it: Lambdas simplify collection operations.

Key Components:

- sort with Comparator
- forEach with Consumer
- filter with Predicate

How YOU should answer: Lambda expressions simplify common collection operations like sorting, filtering, and iteration.

```
Java
// Sort with Lambda
list.sort((a,b) -> a.compareTo(b));

// forEach with Lambda
list.forEach(s -> System.out.println(s));
```

Common Mistakes

- ✗ Using Lambda with non-functional interfaces
- ✗ Complex Lambda expressions
- ✗ Ignoring method references

Expected Interview Questions

- What is Lambda expression?
- Lambda syntax?

- How Lambda connects to functional interfaces?
- Difference between Lambda and anonymous class?

COMPARABLE VS COMPARATOR

13. Comparable vs Comparator ★

Overview

One of the MOST ASKED Java interview topics. Especially for custom sorting, Collections Framework, and real-time data processing.

Topics Covered

- What is Comparable
- What is Comparator
- Comparable vs Comparator ★
- compareTo() Method ★
- compare() Method ★
- Natural Sorting ★
- Custom Sorting ★
- Real-Time Usage ★
- Common Interview Questions

► What is Comparable

What is it: Comparable defines natural ordering using compareTo() inside the class.

Key Components:

- java.lang.Comparable
- compareTo() method
- Single sorting sequence
- Natural order

How YOU should answer: Comparable is used when a class needs to define its natural ordering using compareTo() method.

Java

```
class Student implements Comparable<Student> {  
    int marks;  
  
    public int compareTo(Student s) {  
        return this.marks - s.marks;  
    }  
}
```

```
}  
  
}
```

► What is Comparator

What is it: Comparator defines custom ordering outside the class using `compare()` method.

Key Components:

- `java.util.Comparator`
- `compare()` method
- Multiple sorting sequences
- External class

How YOU should answer: Comparator is used when you need custom or multiple sorting sequences without modifying the original class.

```
Java  
  
Comparator<Student> byName =  
(s1, s2) -> s1.name.compareTo(s2.name);  
  
Collections.sort(students, byName);
```

► Comparable vs Comparator

What is it: Key differences between the two sorting interfaces.

Key Components:

- Comparable — inside class, single sort
- Comparator — outside class, multiple sorts

How YOU should answer: Use Comparable for natural ordering and Comparator for custom or multiple sorting requirements.

Common Mistakes

- ✗ Confusing `compareTo()` and `compare()`
- ✗ Not knowing when to use which

Expected Interview Questions

- Difference between Comparable and Comparator?
- `compareTo()` vs `compare()`?
- When to use Comparable vs Comparator?
- Real-time sorting example?

Feature	Comparable	Comparator
Method	<code>compareTo()</code>	<code>compare()</code>

Package	java.lang	java.util
Location	Inside class	Outside class
Sorting types	Single	Multiple
Modifies class	Yes	No

HASHMAP INTERNAL WORKING

14. HashMap Internal Working ★

Overview

One of the MOST IMPORTANT Java interview topics. Asked in product companies and Java core rounds. Important for Collections Framework and performance understanding.

Topics Covered

- What is HashMap
- Why HashMap is Important ★
- Internal Working of HashMap ★
- Hashing Concept ★
- Collision Handling ★
- hashCode() and equals() ★
- Load Factor
- Rehashing
- Real-Time Usage ★
- Common Interview Questions

► What is HashMap

What is it: HashMap stores key-value pairs using hashing mechanism for O(1) average access time.

Key Components:

- Key-value pairs
- Hashing
- Null key allowed
- Not thread-safe

How YOU should answer: HashMap is a key-value data structure that uses hashing for fast O(1) average-case get and put operations.

```
Map<String,Integer> map = new HashMap<>();
map.put("Kalpesh", 1);
map.put("Java", 2);
System.out.println(map.get("Kalpesh")); // 1
```

► Internal Working

What is it: HashMap internally uses an array of buckets with linked lists for collision handling.

Key Components:

- Array of buckets
- hashCode() determines bucket
- equals() checks key
- LinkedList for collisions
- TreeNode after Java 8 (8+ entries)

How YOU should answer: When you put a key-value pair, HashMap computes hashCode, finds bucket index, and stores using equals() for duplicate checking.

```
Flow

// put("Kalpesh", 1)
// 1. hashCode("Kalpesh") = some number
// 2. index = hashCode % buckets
// 3. Store at index
// 4. If collision -> LinkedList
```

► hashCode and equals Contract

What is it: If two objects are equal, their hashCode must be same.

Key Components:

- Equal objects must have same hashCode
- Override both together

How YOU should answer: hashCode() and equals() must be overridden together to ensure correct HashMap behavior with custom objects.

```
Java

@Override
public int hashCode() { return Objects.hash(name); }

@Override
public boolean equals(Object o) {
    return this.name.equals(((Person)o).name);
}
```


► Collision Handling

What is it: When two keys have same hashCode, they go to same bucket.

Key Components:

- LinkedList in same bucket
- TreeNode after threshold
- Performance degrades with many collisions

How YOU should answer: HashMap handles collisions using chaining with LinkedList, which converts to TreeNode (red-black tree) after 8 entries for performance.

Common Mistakes

- ✗ Not knowing internal working
- ✗ Ignoring hashCode-equals contract
- ✗ Confusing HashMap and Hashtable

Expected Interview Questions

- How HashMap works internally?
- What is hashing?
- What is collision handling?
- hashCode() and equals() contract?
- Difference between HashMap and Hashtable?

★ Most Important HashMap Topics

1. Internal Working
2. hashCode & equals Contract
3. Collision Handling
4. Load Factor & Rehashing

JVM vs JRE vs JDK

15. JVM vs JRE vs JDK ★

Overview

One of the MOST IMPORTANT core Java interview topics. Asked in Java rounds, product companies, framework discussions, and performance discussions.

Topics Covered

- What is JVM
- JVM Architecture ★
- Class Loader ★
- Memory Areas in JVM ★
- Heap vs Stack Memory ★
- JIT Compiler
- JVM vs JRE vs JDK ★
- Platform Independence
- Common Interview Questions

► JVM vs JRE vs JDK

What is it: Three core components of Java platform with different responsibilities.

Key Components:

- JVM — executes bytecode
- JRE — JVM + libraries to run Java
- JDK — JRE + compiler + dev tools

How YOU should answer: JDK is used for development, JRE for running Java applications, and JVM executes the bytecode.

Flow

```
.java file
↓ (javac compiler – part of JDK)
.class (bytecode)
↓ (JVM executes – part of JRE)
Output
```

► JVM Architecture

What is it: JVM consists of Class Loader, Memory Areas, and Execution Engine.

Key Components:

- Class Loader Subsystem
- Runtime Data Areas
- Execution Engine

How YOU should answer: JVM loads, verifies, and executes bytecode using class loader, memory areas, and execution engine.

► Memory Areas in JVM

What is it: JVM divides memory into specific areas for efficient execution.

Key Components:

- Heap — objects and instances
- Stack — method calls and local variables
- Method Area — class metadata
- PC Register — current instruction
- Native Method Stack — native methods

How YOU should answer: JVM manages separate memory areas for objects (Heap), method execution (Stack), and class data (Method Area).

```
Java
```

```
// Heap - objects
Person p = new Person(); // Heap

// Stack - local variables
int age = 25; // Stack
```

► Heap vs Stack Memory

What is it: Key difference between the two main JVM memory areas.

Key Components:

- Heap — dynamic, objects, GC managed
- Stack — fixed, method calls, auto managed

How YOU should answer: Heap stores dynamically created objects managed by GC, while Stack stores method execution data automatically.

Common Mistakes

- ✗ Confusing JRE and JDK
- ✗ Weak JVM memory understanding
- ✗ Not explaining platform independence

Expected Interview Questions

- Difference between JVM, JDK, JRE?
- What is JVM architecture?
- Heap vs Stack memory?
- How Java achieves platform independence?
- What is JIT compiler?

Component	Contains	Purpose
JVM	Execution engine	Runs bytecode
JRE	JVM + libraries	Runs Java apps

JDK	JRE + compiler	Develops Java apps
-----	----------------	--------------------

★ Final Interview Gold — Part 1

1. OOPS Four Pillars + Real-Time Examples
2. String Immutability + String Pool
3. Collections: List vs Set vs Map
4. Exception: Checked vs Unchecked
5. Java 8: Lambda + Streams + Functional Interfaces
6. HashMap Internal Working
7. JVM Memory: Heap vs Stack